

MODUL PRAKTIKUM

COUNTERMEASURES

SECURE CODING



UPH

Universitas Pelita Harapan
FACULTY OF ARTIFICIAL
INTELLIGENCE
AND DATA SCIENCE

Disusun oleh:

Kelvin Wiriyatama, S.Kom.

PROGRAM STUDI INFORMATIKA
FACULTY OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
UNIVERSITAS PELITA HARAPAN
2026



MODUL PRAKTIKUM

Program Studi Informatika

Faculty of AI and Data Science, Universitas Pelita Harapan

Kampus Lippo Village, Tangerang, 15811

Telepon (021) 5460901

DAFTAR ISI

INSTRUCTION	3
1. Setup.....	3
2. Stack Canaries.....	3
3. NX / DEP.....	4
4. ASLR.....	5
5. Summary.....	7
RUBRIC.....	8
REFERENCES.....	8



INSTRUCTION

1. Setup

First confirm you can still reproduce the Lab 2 result. A defense is only convincing once you have seen the attack work without it.

1.1 Build all four configurations and Disable ASLR

From the lab folder, run the setup script (installs checksec and builds every configuration), or just build directly:

```
cd lab3
bash scripts/setup.sh
# ...or build manually:
make all

echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
cat /proc/sys/kernel/randomize_va_space # should print 0
```

1.2 Confirm the Lab 2 exploit still works (unprotected build)

Reproduce the adjacent-variable attack on `auth_demo_unprot`. The 64-byte buffer is followed by the authenticated flag; 64 bytes of padding plus a non-zero value flips it:

```
python3 -c "import sys; sys.stdout.buffer.write(b'A'*64 +
b'\x01\x00\x00\x00')" | ./auth_demo_unprot
```

Expected: ***** ACCESS GRANTED *****. Record this output, it is your “before” baseline for Part 2.

1.3 Run `checksec`

`checksec` gives you a one-line summary of which protections a binary was built with. Use it whenever you are unsure what a build contains:

```
checksec --file=./auth_demo_unprot
```

On the unprotected binary you should see No canary found, NX disabled, and No PIE.

2. Stack Canaries

A stack canary is a known value the compiler places on the stack between the local buffers and the saved return address. Before the function returns, generated code checks whether the canary is unchanged. A contiguous buffer overflow that reaches the return address must pass through the canary first — so it overwrites the canary, the check fails, and the program aborts before the corrupted return address is ever used.

2.1 Rebuild with canary enabled

Only the flags change. `-fstack-protector-all` inserts a canary in every function:

```
make canary
checksec --file=./auth_demo_canary # Canary found
```

2.2 Re-run the exact Lab 2 payload

```
python3 -c "import sys; sys.stdout.buffer.write(b'A'*64 +
b'\x01\x00\x00\x00')" | ./auth_demo_canary
```

Expected result: instead of ACCESS GRANTED, the program prints a message similar to *** stack smashing detected *** and aborts. The same input that won in Lab 2 now triggers a controlled crash.

Record the full output.

2.3 Find the canary in memory

Inspect where the canary sits and how it is checked. Open the binary in gdb:

```
gdb ./vuln_canary
(gdb) disassemble vulnerable
# Look for the prologue line that reads the canary, e.g.:
#   mov     eax, gs:0x14          ; fetch master canary from TLS
#   mov     [ebp-0xc], eax        ; store canary just below saved
EBP
# and the epilogue check:
#   mov     edx, [ebp-0xc]
#   sub     edx, gs:0x14          ; compare; if changed ->
__stack_chk_fail
```

Set a breakpoint just after the canary is stored, run, and read the value:

```
(gdb) break vulnerable
(gdb) run
(gdb) x/1xw $ebp-0xc          # the canary word for this run
(gdb) info registers ebp
```

The low byte of the canary is normally 0x00 (a “null canary”). This is deliberate: it stops string functions from leaking or copying the whole canary as part of a C string. Note the value you see — it changes every run because it is randomized at process start.

2.4 Where the canary still fails

Canaries are strong but not absolute. In your report, explain each of these documented weaknesses in your own words:

- **Information leak:** if the program leaks stack memory (e.g. a format-string bug), the attacker reads the canary and includes the correct value in the payload.
- **Guessing / constant canary:** forking servers that inherit the same canary, or builds with a fixed canary, let an attacker brute-force or reuse it.
- **Overwrite around it:** attacks that corrupt a local variable, a function pointer, or a parameter (without crossing the canary) are not detected at all.

3. NX / DEP

Even if an attacker reaches the return address, they still need somewhere to send execution. The classic move is to inject shellcode into the buffer and jump to it. NX (Non-eXecutable) / DEP (Data Execution Prevention) marks data regions — stack and heap — as non-executable at the hardware level. If the CPU is told to execute an instruction from the stack, it faults.

3.1 Rebuild with NX enabled

In Lab 2 the flag `-z execstack` forced the stack executable. Removing it lets the linker mark the stack NX:

```
make nx
checksec --file=./vuln_nx          # now reports: NX enabled
```

Compare the program headers directly. The `GNU_STACK` segment flags reveal executability — RWE means executable, RW means NX is on:

```
readelf -l ./vuln_unprot | grep -A1 GNU_STACK # RWE
(executable)
readelf -l ./vuln_nx      | grep -A1 GNU_STACK # RW  (non-
executable)
```

3.2 Watch shellcode execution fault

A shellcode-on-the-stack payload that ran in Lab 2 will now fault when the CPU tries to execute stack bytes. Run your Lab 2 shellcode payload (the one that placed shellcode in `buf` and redirected EIP onto the stack) against `vuln_nx`:

```
# Conceptual shape of the Lab 2 stack-shellcode payload:
# [ NOP sled ][ shellcode ][ padding ][ address pointing INTO
the stack ]
# Against vuln_nx the redirect lands on the stack, and the CPU
refuses to
# execute it -> SIGSEGV instead of a shell.
python3 exploit_shellcode.py | ./vuln_nx
```

Expected: a segmentation fault at the moment of execution, not a shell. Capture the crash. Confirm in `gdb` that the faulting address lies inside the stack range from `info proc mappings`.

NX **does not** stop the overflow or the return-address overwrite, it only the execution of injected code. The `win()` redirect from Lab 2 (jumping to existing, already-executable program code) is unaffected by NX. That distinction is the whole point of the next note.

3.3 The bypass: Return-Oriented Programming

Because NX only blocks **injected** code, attackers reuse code that is **already executable**. **Return-Oriented Programming (ROP)** chains together short instruction sequences (“gadgets”) that already exist in the program or its libraries, each ending in a `ret`, to perform arbitrary computation without injecting a single new instruction. You will not build a full ROP chain in this lab — jumping to the single existing `win()` function (as in Lab 2) is the simplest one-gadget illustration that NX cannot stop. Note this in your report as the reason NX alone is insufficient.

4. ASLR

Both the shellcode jump and the `win()` redirect depend on the attacker knowing an **address** in advance. **Address Space Layout Randomization (ASLR)** attacks that assumption by

randomizing the load addresses of the stack, heap, and shared libraries each time the program starts. Hardcoded addresses from Lab 2 stop being reliable.

4.1 Re-enable ASLR

```
echo 2 | sudo tee /proc/sys/kernel/randomize_va_space
cat /proc/sys/kernel/randomize_va_space # should print 2
```

4.2 Observe addresses moving

Run a PIE build three times and watch the stack and library base addresses change on each run. (PIE (Position-Independent Executable) is the binary-side feature that lets the loader randomize the main program too.)

```
for i in 1 2 3; do echo "--- run $i ---"; \
  setarch -R cat /proc/self/maps >/dev/null; \
  ./vuln_pie </dev/null & PID=$!; \
  grep -E 'stack|libc' /proc/$PID/maps 2>/dev/null | head; \
  wait $PID 2>/dev/null; done
```

A simpler, reliable demonstration: print the address of a stack variable across runs, and compare /proc/self/maps snapshots:

```
# Three snapshots of the loader's own mapping - note the
differing base addresses:
for i in 1 2 3; do echo "run $i:"; cat /proc/self/maps | grep -
E 'stack|libc' | head -2; done
```

4.3 Why the hardcoded Lab 2 address now fails

In Lab 2 you hardcoded the address of win() (a fixed value like 0x080491b6) because the -no-pie binary always loaded at the same place. Against a PIE build with ASLR on, win() sits at a different address every run, so the fixed payload misses. Try your Lab 2 win() payload against vuln_pie several times and record how many succeed.

NOTE: On 32-bit systems ASLR has relatively low entropy (only a limited number of bits are randomized) so a brute-force attack against a process that restarts on crash (e.g. a forking server) can defeat it in a feasible number of attempts. 64-bit systems have far more entropy and are correspondingly harder. State this trade-off in your report.

4.4 Defense combine

Notice that ASLR and NX attack different links in the chain: NX stops you **running** injected code, ASLR stops you **finding** the code you want to reach. Defeating both at once (e.g. a leak to beat ASLR plus ROP to beat NX) is what real modern exploits must do, which is precisely why layering them works.

5. Summary

Pull the session together by completing the matrix below in your report. For each countermeasure, state which stage of an attack it disrupts and the documented technique that defeats it. (The “attack stage” column refers to the Eternal War in Memory model: corrupt a pointer → read/write memory → hijack control flow.)

Countermeasure	Compiler flag / setting	Stage it disrupts	Known bypass (fill in)
Stack canary (SSP)	<code>-fstack-protector-all</code>	Detects RA tampering before return	_____
NX / DEP	<code>(omit -z execstack)</code>	Blocks execution of injected code	_____
ASLR + PIE	<code>-fpie -pie / randomize_va_space=2</code>	Hides addresses the attacker needs	_____



MODUL PRAKTIKUM

Program Studi Informatika

Faculty of AI and Data Science, Universitas Pelita Harapan

Kampus Lippo Village, Tangerang, 15811

Telepon (021) 5460901

RUBRIC

Component	Weight	Criteria
Part 1 baseline	10%	Unprotected exploit reproduced (ACCESS GRANTED), ASLR confirmed off, checksec output captured.
Part 2 canary	30%	Stack-smashing abort shown on canary build; gdb screenshot of canary value; three weaknesses explained in own words.
Part 3 NX	25%	readelf GNU_STACK comparison (RWE vs RW); shellcode fault captured; correct explanation of why ROP/win() is unaffected.
Part 4 ASLR	25%	Addresses shown changing across ≥ 2 runs; tally of hardcoded-payload hits/misses; correct low-entropy explanation.
Part 5 + quality	10%	Defense matrix completed correctly; clear structure, readable screenshots, original wording.

REFERENCES

- Aspinall, D. Secure Programming — Lecture 5 (Memory Corruption III: Countermeasures). University of Edinburgh.
- Szekeres, L., Payer, M., Wei, T., & Song, D. (2013). SoK: Eternal War in Memory. IEEE Symposium on Security and Privacy.
- Cowan, C., et al. (1998). StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks. USENIX Security.
- Shacham, H. (2007). The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86). ACM CCS. [ROP]
- PaX Team. Address Space Layout Randomization (ASLR) design documentation.
- CISA / NSA / FBI (2023). The Case for Memory Safe Roadmaps.